

Helm Validation & Testing

Make Charts Safe, Predictable, and CI-Ready

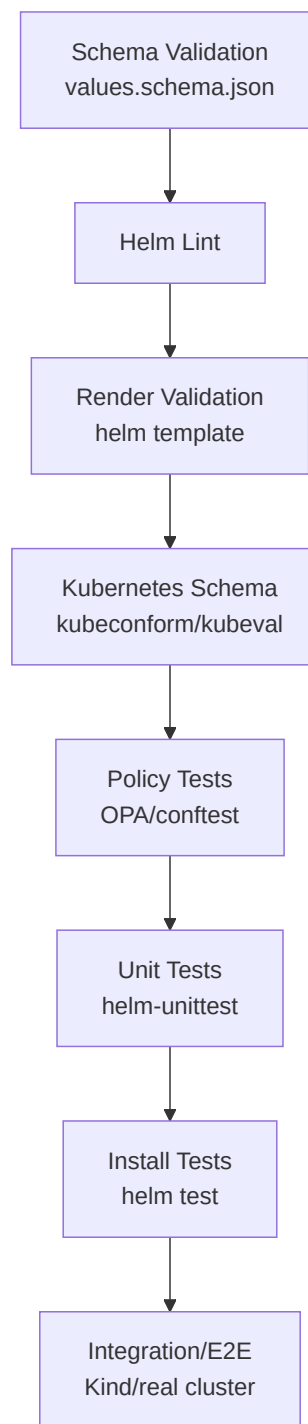
You'll learn:

- Validation layers: schema, linting, rendering, kube checks
- Unit, integration, and e2e testing strategies
- Tools: `helm lint`, `values.schema.json`, `helm test`, `helm-unittest`, `ct`, `kubeconform`, `kube-linter`, `kube-score`, `conftest` (OPA)
- Golden/snapshot tests, diffs, and upgrade safety
- CI pipelines and local dev workflows

Why Validate & Test Charts?

- Prevent runtime failures **before** hitting the cluster
- Catch breaking changes across **upgrades**
- Enforce **API compatibility** and **policy**
- Build **trust**: reusable, maintainable charts others can consume

Goal: *Deterministic rendering + policy compliance + safe rollout.*



Validation Pyramid (Mental Model)

Start cheap/fast at the top; progress to costlier (cluster) checks.

`values.schema.json` (JSON Schema)

- Put at chart root to **validate user values**
- Enforced by `helm install/upgrade` and `helm lint`
- Prevents invalid types, ranges, or missing required fields

Example

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "title": "myapp values",
  "type": "object",
  "properties": {
    "replicaCount": {"type": "integer", "minimum": 1, "maximum": 50},
    "image": {
      "type": "object",
      "properties": {
        "repository": {"type": "string", "minLength": 1},
        "tag": {"type": "string", "pattern": "^[a-zA-Z0-9_.-]+$"}
      },
      "required": ["repository"]
    },
    "service": {
      "type": "object",
      "properties": {
        "type": {"enum": ["ClusterIP", "NodePort", "LoadBalancer"]},
        "port": {"type": "integer", "minimum": 1, "maximum": 65535}
      },
      "required": ["type", "port"]
    }
  },
  "required": ["image", "service"],
  "additionalProperties": true
}
```

Helm Lint

```
helm lint ./chart  
helm lint ./chart --strict
```

Catches:

- Missing/invalid `chart.yaml` fields
- Malformed templates (render errors)
- Fails on schema violations (if `values.schema.json` exists)

`--strict` elevates warnings to errors.

Render Validation (No Cluster Changes)

```
# Render with specific values (per env)
helm template myapp ./chart -f values.yaml -f values-prod.yaml \
  --kube-version=1.30 --namespace prod > rendered.yaml
```

Use cases:

- Check **conditional blocks** resolve correctly
- Verify generated object names/labels/selectors
- Produce artifacts for code review (golden files)

Kubernetes Schema Validation

Validate rendered YAML against Kubernetes OpenAPI.

kubeconform (fast/parallel)

```
kubeconform -summary -strict -kubernetes-version 1.30 rendered.yaml
```

kubeval (classic)

```
kubeval --strict --kubernetes-version 1.30 rendered.yaml
```

Add CRD schemas via `--schema-location` or skip custom kinds when needed.

Policy & Security Checks

kube-linter (best practices)

```
kube-linter lint rendered.yaml
```

kube-score (static analysis)

```
kube-score score rendered.yaml
```

conftest (OPA/Rego policies)

```
# policy.rego evaluates rendered manifests  
conftest test rendered.yaml
```

Examples to enforce:

- No `:latest` images, resource limits set
- Drop `NET_ADMIN`, `runAsNonRoot`, disallow `hostPath`
- Enforce standard labels/annotations

Unit Testing Templates (helm-unittest)

Install plugin

```
helm plugin install https://github.com/helm-unittest/helm-unittest
```

Folder layout

```
chart/  
  templates/  
  values.yaml  
  tests/  
    deployment_test.yaml
```

Example: tests/deployment_test.yaml

```
suite: Deployment rendering
templates:
  - templates/deployment.yaml
values:
  - values.yaml
  - tests/values/minimal.yaml
tests:
  - it: sets replicaCount
    asserts:
      - equal:
          path: spec.replicas
          value: 3
  - it: injects image tag
    set:
      image.tag: "1.2.3"
    asserts:
      - equal:
          path: spec.template.spec.containers[0].image
          value: "myrepo/myimg:1.2.3"
  - it: requires service port in env
    set:
      service.port: 8080
    asserts:
      - exists:
          path: spec.template.spec.containers[0].ports[0]
```

Snapshot (Golden) Tests

- Render templates with canonical inputs → store as **golden YAML**
- On change, diff vs. golden to catch **unintended changes**

```
mkdir -p golden
helm template myapp ./chart -f tests/values/golden.yaml > golden/myapp.yaml
# later in CI
helm template myapp ./chart -f tests/values/golden.yaml > out.yaml
diff -u golden/myapp.yaml out.yaml
```

Combine with `helm-diff` plugin for upgrade comparisons.

Upgrade Safety (helm-diff)

```
helm plugin install https://github.com/databus23/helm-diff  
  
# Show changes between revisions or values  
helm diff upgrade myapp ./chart -n demo -f values.yaml -f values-prod.yaml
```

Check for:

- Resource recreation vs. in-place patches
- Immutable fields changes (e.g., PVC `storageClassName`)
- Service type/selector drift

helm test (Install Tests)

- Runs Kubernetes **Jobs/Pods** marked with `helm.sh/hook: test`
- Verifies app health post-install/upgrade

Example

```
apiVersion: v1
kind: Pod
metadata:
  name: "{{ include \"myapp.fullname\" . }}-test"
  annotations:
    "helm.sh/hook": test
spec:
  restartPolicy: Never
  containers:
    - name: smoke
      image: curlimages/curl
      args: ["-f", "http://{{ include \"myapp.fullname\" . }}:{{ .Values.service.port }}/healthz"]
```

Run

```
helm test myapp -n demo --logs
```

Keep tests fast & idempotent; clean up with hook delete policies if needed.

Testing Hooks & Lifecycle

Use hook weights and delete policies to orchestrate tests:

```
metadata:  
  annotations:  
    "helm.sh/hook": pre-install,post-upgrade,test  
    "helm.sh/hook-weight": "-5"  
    "helm.sh/hook-delete-policy": hook-succeeded
```

Patterns:

- Pre-install DB migrations
- Post-upgrade smoke checks
- On-failure retain resources for debugging

API Compatibility & **.Capabilities**

Guard templates for cluster versions:

```
apiVersion: {{- if .Capabilities.APIVersions.Has "networking.k8s.io/v1" -}}networking.k8s.io/v1{{- else -}}extensions/v1beta1{{- end -}}
```

Validate against multiple versions in CI:

```
helm template myapp ./chart --kube-version 1.26  
helm template myapp ./chart --kube-version 1.30
```

Catch deprecations (e.g., **Ingress** v1beta1) early.

Required Values & Failing Fast

Use template guards to stop bad renders:

```
{{- required "image.repository is required" .Values.image.repository -}}
```

Sprig `fail` to abort with message:

```
{{- if not .Values.tls.enabled -}}  
{{- fail "TLS must be enabled in production" -}}  
{{- end -}}
```

Prefer schema for types; use `required` / `fail` for **contextual rules**.

Data Shaping & Helpers

- Normalize labels/annotations in `_helpers.tpl`
- Add stable names via `printf` + `trunc` + `trimSuffix`
- Use `tpl` to re-evaluate templated strings from values when needed

```
{{- define "labels.standard" -}}  
app.kubernetes.io/name: {{ include "name" . }}  
app.kubernetes.io/instance: {{ .Release.Name }}  
helm.sh/chart: {{ .Chart.Name }}-{{ .Chart.Version }}  
{{- end -}}
```

Testing tip: assert these helpers render identically across resources.

Chart Testing (ct) — Lint & Install in CI

Install: <https://github.com/helm/chart-testing>

Core commands:

```
ct list-changed  
ct lint --config ct.yaml  
ct install --config ct.yaml
```

`ct.yaml` example

```
chart-dirs: ["charts"]  
helm-extra-args: "--timeout 10m"  
check-version-increment: true  
validate-maintainers: false  
upgrade: true
```

`ct install` spins up a **Kind** cluster by default (GitHub Action) and installs charts to verify they work.

Example GitHub Actions CI

```
name: chart-ci
on: [pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: azure/setup-helm@v4
      - name: Lint (helm)
        run: |
          helm lint charts/myapp --strict
      - name: Render & Validate
        run: |
          helm template myapp charts/myapp -f charts/myapp/values.yaml --kube-version 1.30 > out.yaml
          curl -L https://github.com/yannh/kubeconform/releases/latest/download/kubeconform-linux-amd64.tar.gz | tar xz
          ./kubeconform -summary -strict out.yaml
      - name: Security Lints
        run: |
          curl -sSfL https://raw.githubusercontent.com/stackrox/kube-linter/main/install.sh | sh -s -- -b .
          ./kube-linter lint out.yaml || true
      - name: Chart Testing (ct)
        uses: helm/chart-testing-action@v2
        with:
          command: lint-and-install
```

Local Dev Loop

```
# 1) Fast feedback
helm lint ./chart && \
helm template myapp ./chart --kube-version 1.30 > out.yaml && \
kubecoreform -summary -strict out.yaml

# 2) Unit tests
helm unittest ./chart

# 3) Install smoke test on Kind
kind create cluster --name helm-dev || true
helm upgrade --install myapp ./chart -n dev --create-namespace
helm test myapp -n dev --logs
```

Testing Dependencies & Subcharts

- Toggle with `condition` flags in values during tests
- Provide stub values for required subcharts
- Use `ct install --upgrade` to verify upgrades across dependency versions

```
# Chart.yaml
dependencies:
- name: redis
  version: "~17.0.0"
  repository: "https://charts.bitnami.com/bitnami"
  condition: redis.enabled
```

```
# tests/values/minimal.yaml
redis:
  enabled: false
```

Test Data & Fixtures

- Keep `tests/values/` for scenario-specific values
- Name fixtures by behavior: `minimal.yaml`, `prod-like.yaml`, `ingress.yaml`
- Avoid secrets; use placeholders or SOPS in CI secrets store

```
# tests/values/prod-like.yaml
resources:
  limits:
    cpu: "500m"
    memory: "512Mi"
ingress:
  enabled: true
  className: "nginx"
```

Common Failure Patterns (and Tests)

- **Selector drift:** Service label selectors don't match Deployment labels → add unit assert on selectors
- **Immutable fields:** PVC size/class changes → test diffs and document migration steps
- **Name truncation:** Names >63 chars → assert `metadata.name` fits with `trunc`
- **Missing CRDs:** Helm installs CRDs out of order → validate CRDs exist or separate CRD chart

Releasing with Confidence

Checklist:

-  `values.schema.json` comprehensive
-  `helm lint --strict` clean
-  Rendered YAML passes kubeconform & policy linters
-  Unit tests (helm-unittest) green
-  `ct install` succeeds on fresh Kind
-  Upgrade path verified with `helm diff` + `ct install --upgrade`

Hands-On Lab (15–20 min)

1. Add schema & break it on purpose

```
cp examples/values.schema.json charts/myapp/values.schema.json  
helm lint charts/myapp --strict # should fail when broken
```

2. Add a helm-unittest suite

```
mkdir -p charts/myapp/tests  
cp examples/deployment_test.yaml charts/myapp/tests/  
helm unittest charts/myapp
```

3. Render & validate against K8s

```
helm template myapp charts/myapp -f charts/myapp/values.yaml > out.yaml  
kubconform -summary -strict out.yaml
```

4. Smoke test on Kind

```
kind create cluster --name lab || true  
helm upgrade --install myapp charts/myapp -n lab --create-namespace  
helm test myapp -n lab --logs
```


Key Takeaways

- Treat charts like **software**: types, tests, CI, reviews
- Shift-left with schema & unit tests
- Validate rendered YAML against **real K8s schemas**
- Use **policy** to encode org standards
- Practice safe upgrades with diffs & e2e checks